

Smart MCQ Solver Challenge

DLGenAI Project Report

Name: Shikhar Singh | **Roll No.:** 23f2003771 | **WandB Project:** qubyt/dlgenai-project-26t2 | **GitHub:** 23f2003771/dl-genai-project-26-t2 | **Kaggle Competition:** Smart MCQ Solver Challenge | **Best Public Score:** 0.75893 | **Email:** 23f2003771@ds.study.iitm.ac.in

1. Abstract

This report details three independent approaches to the Smart MCQ Solver Challenge, a five-option multiple-choice task evaluated by MAP@3. The models developed include: **(1)** a custom neural option-scorer trained from scratch on Word2Vec embeddings, **(2)** a Retrieval-Augmented Generation (RAG) pipeline utilizing a quantized LLM and FAISS, and **(3)** a Logistic Regression baseline. All models surpassed the 0.73 MAP@3 threshold, with the RAG pipeline achieving the highest score of **0.75893**.

2. Introduction

Problem Statement. The challenge involves predicting the top three most likely correct answers for a given question with five options (A–E). The evaluation metric is Mean Average Precision at 3 (MAP@3), rewarding the correct answer's early placement in the top-3 ranking.

Project Objective. The objective was to implement and compare three distinct modeling strategies—a custom model, a pre-trained/RAG model, and a baseline—evaluating them under a constrained compute budget using common metrics.

3. Dataset & Preprocessing

Dataset Description. The training set comprises 2,000 labeled questions (mostly physics/astronomy), and the test set has 500 unlabeled questions. Each entry contains a prompt and five options (A–E), with no missing values.

EDA. The answer distribution is mildly imbalanced (B: 490, C: 459, A: 369, D: 358, E: 324), so we report weighted F1 rather than macro F1. Prompt length averages 18.1 words (std 6.8, range 3–51), indicating mostly short, direct questions.

Preprocessing. All text fields were lowercased, stripped of non-alphanumeric characters, and whitespace-tokenized via a single clean_text pass. No stemming/lemmatization was applied, since our downstream embeddings (Word2Vec and the RAG embedding model) handle raw tokens adequately without it.

4. Tokenization / Feature Representation Strategy

Two representation strategies were used, matched to what each model needed:

Word2Vec (from-scratch NN and Logistic Regression). A Word2Vec model (vector_size=100, window=5) was trained on all texts, yielding a 2,973-word vocabulary. Texts are represented by the mean of their word vectors.

BGE-small-en-v1.5 (RAG pipeline). For retrieval, we used HuggingFace's BAAI/bge-small-en-v1.5 sentence-embedding model with normalized embeddings, indexed via FAISS.

Experimental comparison. I also evaluated BGE-large-en-v1.5 and BGE-M3 as drop-in replacements for the retrieval embedding model. Both reduced the final MAP@3 score noticeably compared to BGE-small-en-v1.5, so the smaller model was retained for both knowledge bases in the final pipeline.

5. Modeling & Experimentation

Three models were built, each tracked as a separate WandB run under the dlgenai-project-26t2 project.

5.1 Model 1 (From Scratch): MCQScorer — Word2Vec + MLP Option Scorer

Architecture. MCQScorer is a custom PyTorch module. For each option, the question vector and option vector (100-d each, from Word2Vec) are concatenated into a 200-d input and passed through an MLP (200→256→128→64→1) with ReLU activations and dropout (0.2, 0.25), producing one scalar plausibility score per option. Scores across the five options are combined via softmax at inference time to produce a ranking.

Design Rationale. Framing this as a per-option scorer rather than a direct 5-way classifier lets the network evaluate each option independently and produces a natural top-3 ranking, aligning directly with the MAP@3 objective.

Training Details. Trained for 50 epochs (batch size 64) with Adam (lr=0.001) and CrossEntropyLoss over the five option scores, checkpointing on best validation F1. Validation accuracy (0.8975) is higher than training accuracy (0.7906) in this run because dropout remains active during the training pass (model.train()) and is disabled for validation (model.eval()), which is standard behavior rather than an anomaly. A learning-rate sweep confirmed lr=0.001 as the best of the values tried. I additionally explored removing dropout entirely, adding extra hidden layers, and varying hidden-layer width; all such variants underperformed the reported configuration and were not carried forward into the formal comparison.

5.2 Model 2 (Pretrained): RAG Pipeline

Architecture. Retrieves top-3 similar chunks from a FAISS index of ~200 physics/astronomy Wikipedia articles and top-2 similar training examples. These are inserted into a prompt with the question/options. The prompt is fed to **google/gemma-4-12B** (4-bit NF4 quantized), generating text parsed for A–E letters to form a ranked list.

Salient Points. This is the only model that uses external knowledge beyond the training set, making it well-suited as our pretrained model — performance here depends on retrieval quality and prompt design as much as on the LLM's own knowledge.

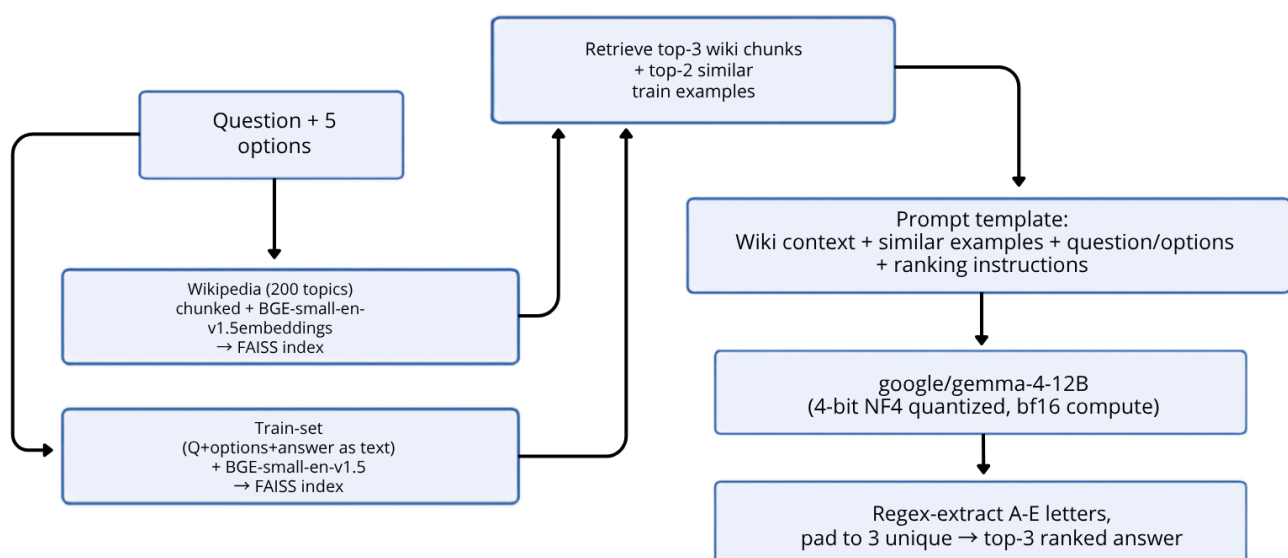
Backbone LLM Selection. In addition to Gemma-4-12B, I evaluated DeepSeek-R1-Distill-Qwen-14B as an alternative backbone under the same retrieval and prompting setup, achieving 0.75062 MAP@3 — ahead of the Logistic Regression baseline but below Gemma-4-12B (0.75893). Gemma-4-12B was retained as the final backbone on this basis.

Quantization. 4-bit NF4 quantization (via bitsandbytes) allowed a ~12B-parameter model to run within Kaggle's dual-T4 GPU environment.

Deployment. To make the pipeline usable beyond the notebook, I deployed it as a standalone Gradio web application on Modal, using an on-demand L4 GPU with memory/GPU snapshotting for fast cold starts.

Live demo: <https://singh-595647--dlgenai-model-ragapp-ui.modal.run>

RAG Pipeline (Retriever + Quantized LLM)



5.3 Model 3 (Model of Choice): Logistic Regression

Architecture. A multinomial Logistic Regression (scikit-learn, $C=0.3$, L2 penalty, $\text{max_iter}=1000$) trained on a 600-d feature vector: the question's Word2Vec embedding concatenated with all five options' embeddings. Unlike the other two models, this is a direct 5-way classifier; ranking is obtained by sorting `predict_proba` output.

Salient Points. Logistic Regression was chosen as my model of choice specifically for its simplicity: it serves as a fast, interpretable lower bound, letting us judge how much the from-scratch NN and RAG pipeline actually gain from additional model capacity and external knowledge on the same features.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

We report two metric families. **Validation accuracy / weighted F1** (WandB-tracked) are computed on top-1 predictions. **Kaggle MAP@3**, the competition's scoring metric, gives partial credit when the true answer appears anywhere in the top-3 ranked predictions. The two are related but not directly comparable, since they credit different behavior.

6.2 Comparative Table

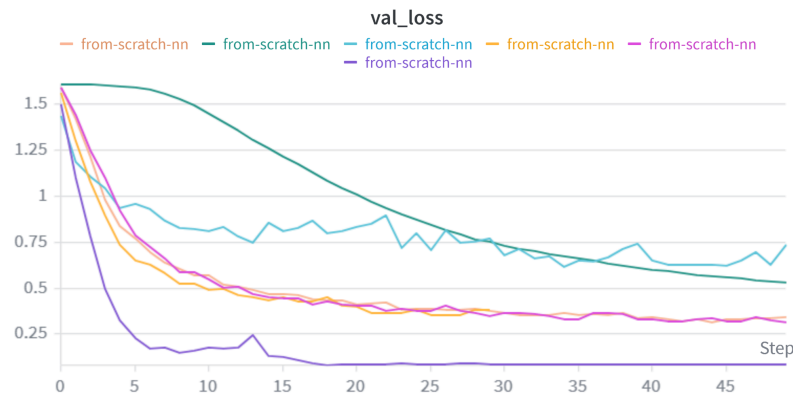
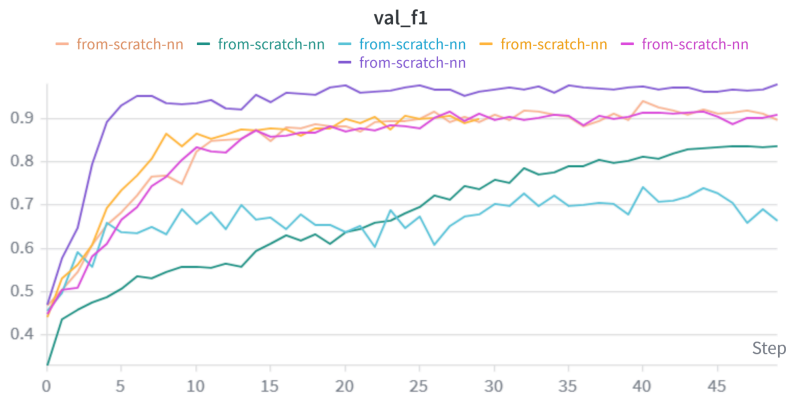
Model	Type	Val Accuracy	Val F1	Train Accuracy	Train F1	Kaggle MAP@3
MCQScorer (Word2Vec + MLP)	From scratch	0.8975	0.8971	0.7906	0.7905	0.73316
RAG (Gemma-4-12B + retrieval)	Pretrained	0.9800	0.9799	–	–	0.75893
Logistic Regression ($C=0.3$)	Model of choice	0.9975	0.9975	0.9988	0.9988	0.74688

(RAG has no train-set row since it performs no training — it is used purely for inference; All three models individually exceed the 0.73 MAP@3 pass threshold.)

6.3 Training Performance

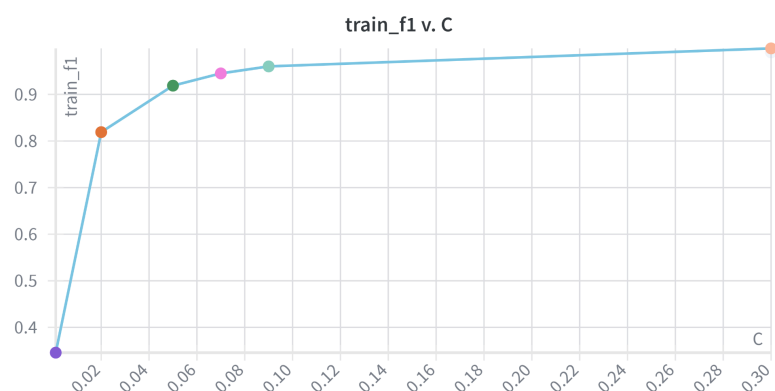
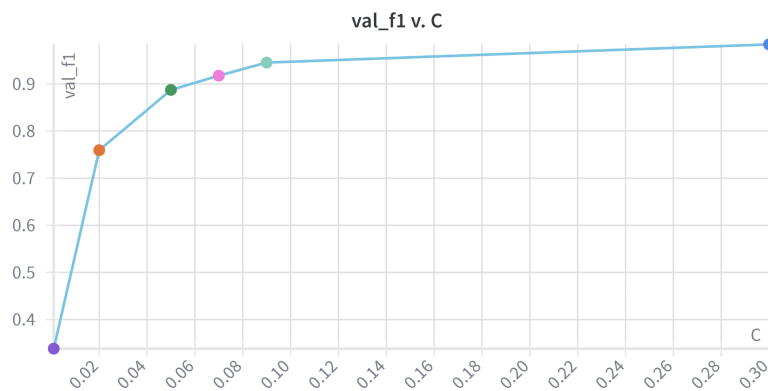
From-scratch NN (Top 5 variants):

Most variants converged stably. A plateau observed in one variant was tied to a learning rate issue ($\text{lr}=0.01$ destabilized training) and change in dropout layers.



Logistic Regression (Regularization Impact):

Varying the regularization parameter C revealed that under-regularization (low C) hampered performance. Both training and validation metrics stabilized near $C=0.1$. $C=0.3$ was ultimately selected based on the highest leaderboard score (0.74688).



6.4 Kaggle Performance

Final leaderboard score: **0.75893** MAP@3, achieved via the RAG pipeline. All three models individually cleared the 0.73 pass threshold when evaluated independently and on the public leaderboard.

7. Conclusion & Future Work

Key Learnings. Building all three models side-by-side made the trade-offs concrete: the from-scratch model required the most training engineering for the lowest absolute score; the RAG pipeline required the most infrastructure (quantization, dual FAISS indices, prompt design, deployment) for the best score; and the Logistic Regression baseline, built with far less effort, came within a couple of points of the from-scratch model — a useful reference point for judging how much capacity and external knowledge actually contribute on this dataset.

Challenges Faced. The small training dataset (2,000 rows) necessitated reliance on pre-trained components. Compute constraints (dual-T4 GPUs) limited the size of the LLM backbones that could be tested.

Areas for Improvement.

- Ensemble the three models' top-3 rankings (rank fusion) rather than submitting a single model's output.
- Fine-tune a smaller transformer encoder (e.g., DeBERTa-v3-small) directly on the MCQ scoring task as an additional comparison point.
- Evaluate larger LLM backbones (26B+ parameters) given additional GPU/VRAM budget beyond the dual-T4 environment used here.

8. References

Tools & Documentation

- Gensim (Word2Vec) — <https://radimrehurek.com/gensim/>
- FAISS — <https://faiss.ai/>
- Hugging Face Transformers — <https://huggingface.co/docs/transformers>
- BAAI/bge-small-en-v1.5 model card — <https://huggingface.co/BAAI/bge-small-en-v1.5>
- LangChain — <https://python.langchain.com/>
- DeepSeek-R1-Distill-Qwen-14B model card — <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B>
- Weights & Biases — <https://docs.wandb.ai/>
- Modal — <https://modal.com/docs>